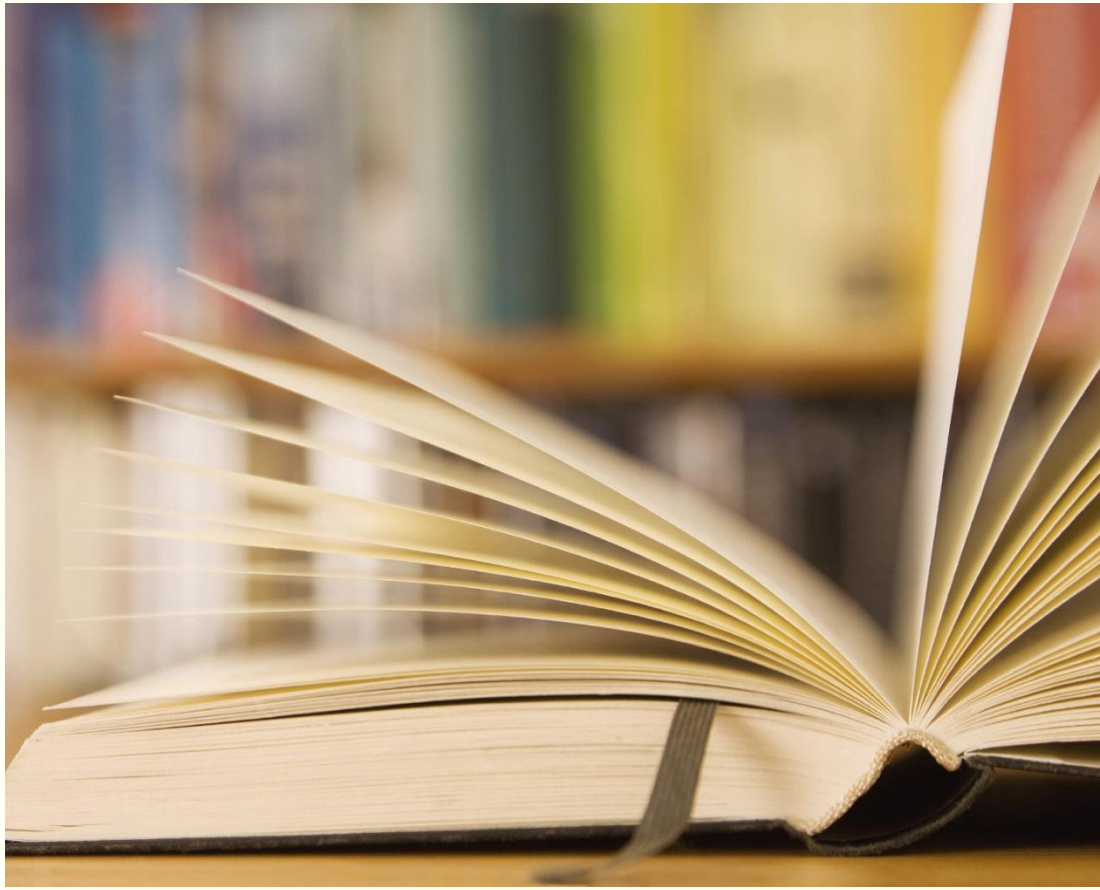
FINITE-STATE AUTOMATA

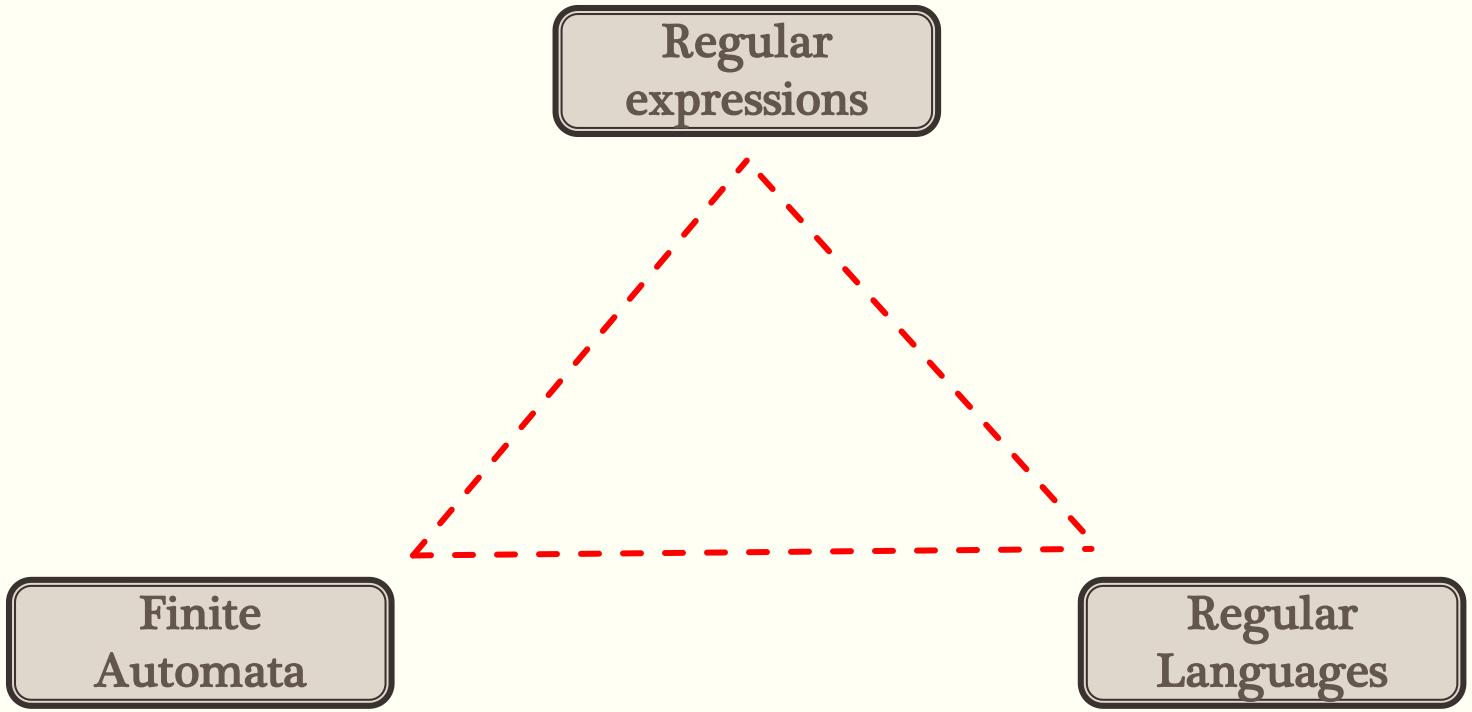
13.01.2026



Introduction

- Finite-state automata are the theoretical foundation of a good deal of the computational work.
- Regular expression can be implemented as a finite-state automaton.
- A regular expression is one way of characterizing a particular kind of formal language called a regular language.
- Both regular expressions and finite-state automata can be used to described regular languages.

The relation among these three theoretical constructions



Introduction

- An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).
- Mathematically, an automaton can be represented by a 5-tuple

  $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Contd…

- The states are represented by **vertices**.
- The transitions are shown by labeled **arcs**.
- The initial state is represented by an **empty incoming arc**.
- The final state is represented by **double circle**.
  - Q = {a, b, c},
  - Σ = {0, 1},
  - q_0 = {a},
  - F = {c}

Finite Automata – two types

- Deterministic Finite State Automaton (DFA) – With each input, **one could easily determine the next state to which the machine would navigate.** I.e. it is **deterministic.** It is to be understood that the **number of states are finite** with DFA and hence is called Deterministic Finite State Automaton.

- Non Deterministic Finite State Automata (NFSA) – For a particular input, the **machine may move to any of the many states.** One cannot confine/restrict NFA to **navigate to any one particular state.** I.e. moves are non deterministic. So, we term it as NFA. But, **number of states are finite** ☺ Remember this point.

Contd…

DFA

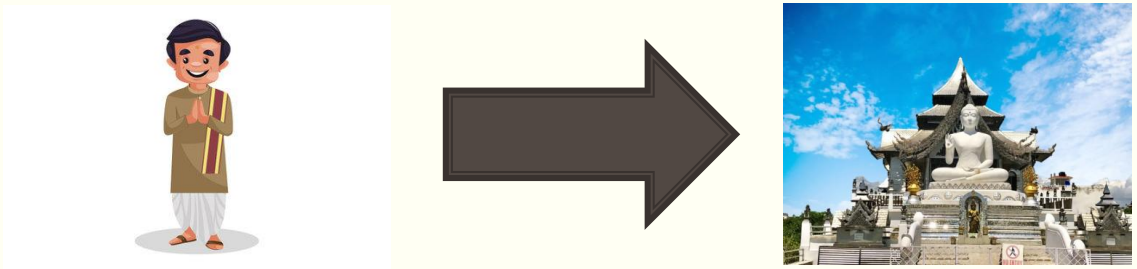
- Given a current state w.k what the next state will be.
- It has only one unique next state
- It has no choices or randomness
- It is simple and easy to design

NFA

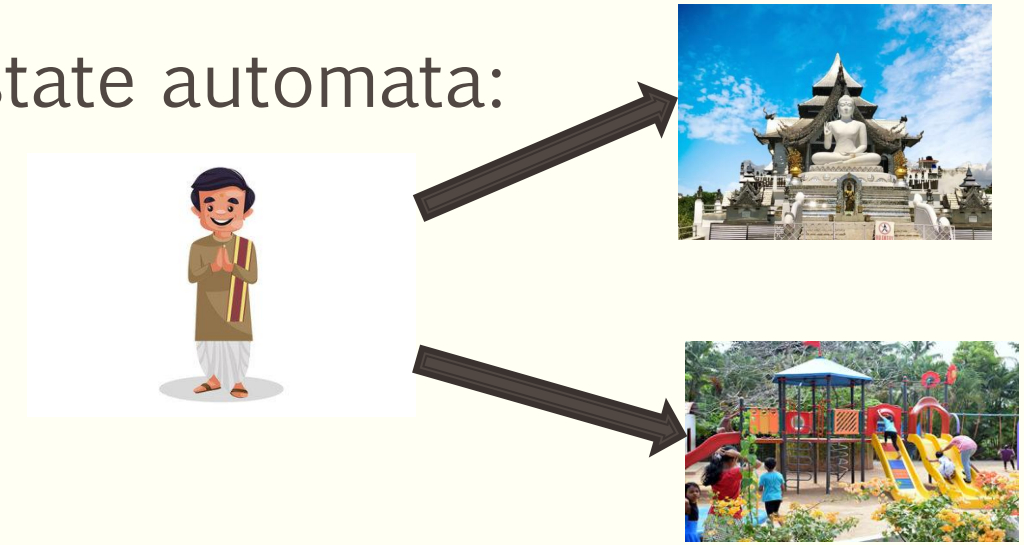
- In NFA, Given the current state there could be multiple next states.
- The next state may be chosen at random.
- The next states may be chosen in parallel.
- ε (No input)

TYPES of FSA

- Deterministic finite state automata:

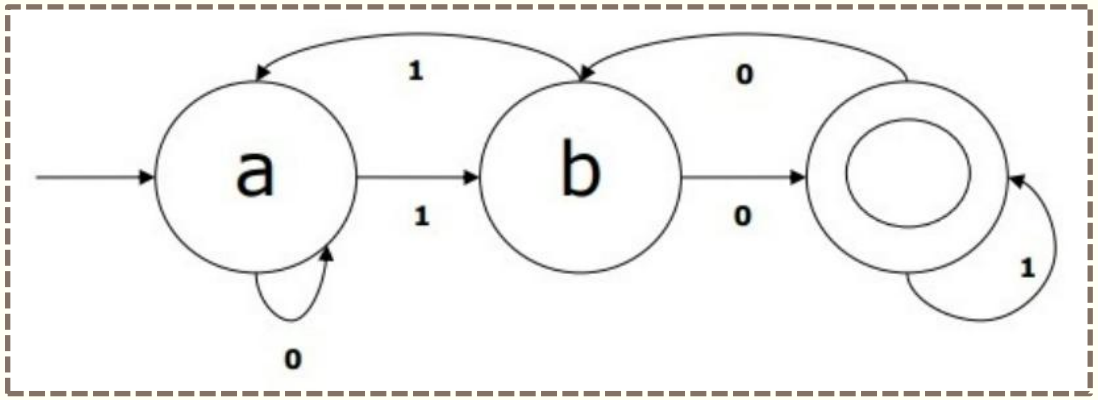


- Non-Deterministic Finite state automata:



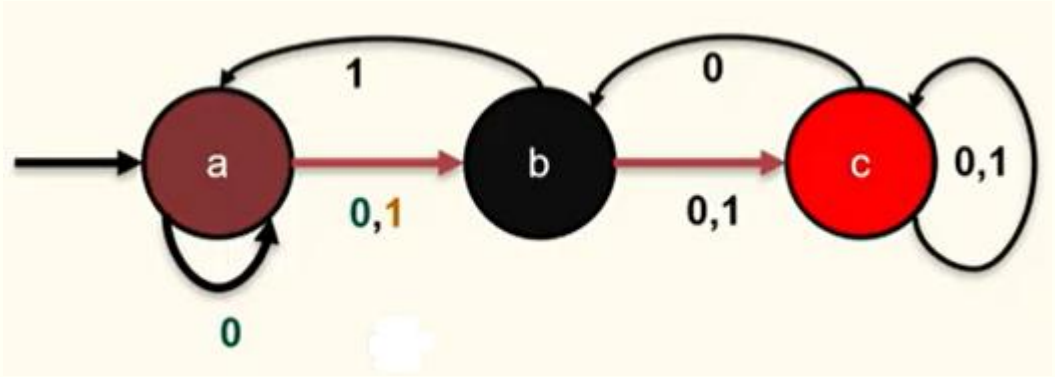
DFA – Deterministic Finite State Automata

Current State	Next State for Input 0	Next State for Input 1
a	a	b
b	c	a
c	b	c



NDFA or NFA – Non-Deterministic Finite State Automata

Current State	Next State for Input 0	Next State for Input 1
a	a,b	b
b	c	a,c
c	b,c	c

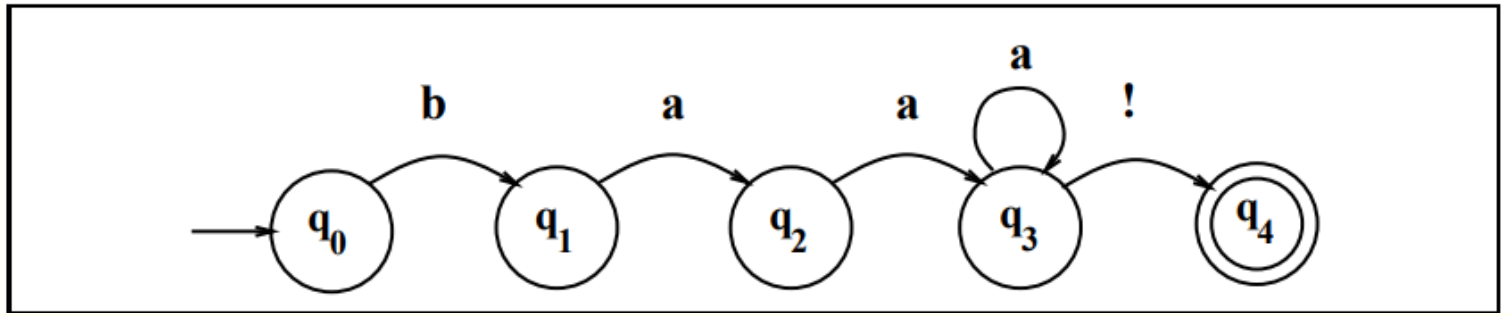


Examples: Conversion of RE to FSA

- ba^*b – bb, bab, baab, baaab …
- (a+b)c – ac or bc
- $a(bc)^*$ – a, abc, abcbc …

Using an FSA (DFA) to Recognize Sheeptalk

baa!
baaa!
baaaa!
baaaaa!
baaaaaa!



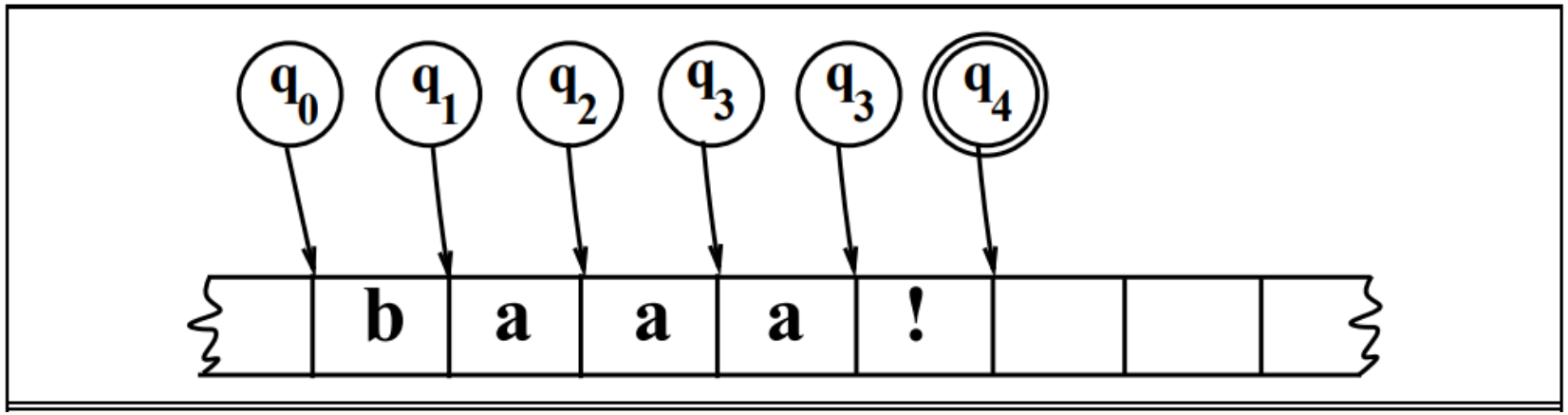
A finite-state automaton for talking sheep

state-transition table

	Input		
State	b	a	!
0	1	Ø	Ø
1	Ø	2	Ø
2	Ø	3	Ø
3	Ø	3	4
4:	Ø	Ø	Ø

- 4 with a colon to indicate that it's a final state
- ϕ indicates an illegal or missing transition

Tracing the execution of FSA #1 on some sheep talk

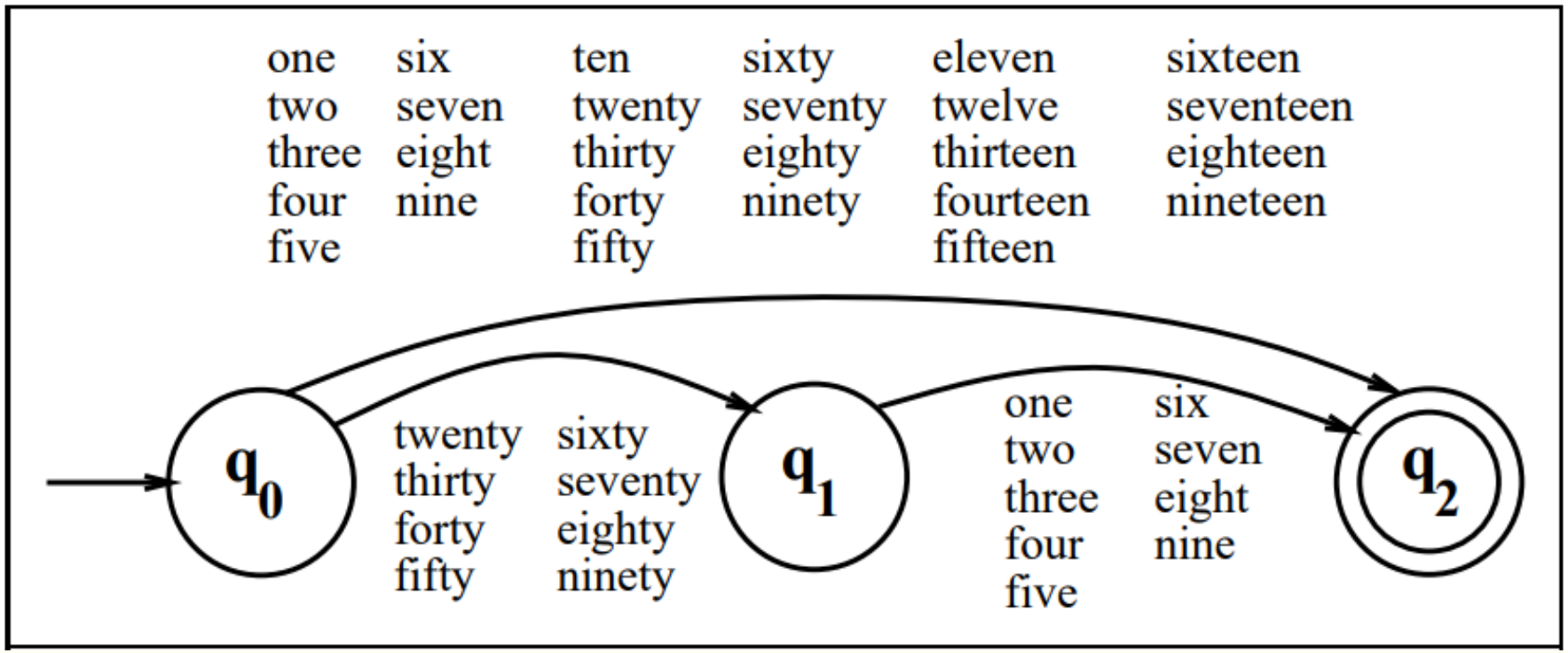


Contd…

- A formal language is a set of strings, each string composed of symbols from a finite symbol-set called an alphabet (the same alphabet used above for defining an automaton!).
- The alphabet for the sheep language is the set $\Sigma = \{a, b, !\}$.
- Given a model m (such as a particular FSA), we can use L(m) to mean "the formal language characterized by m".
- So the formal language defined by our sheeptalk automaton m

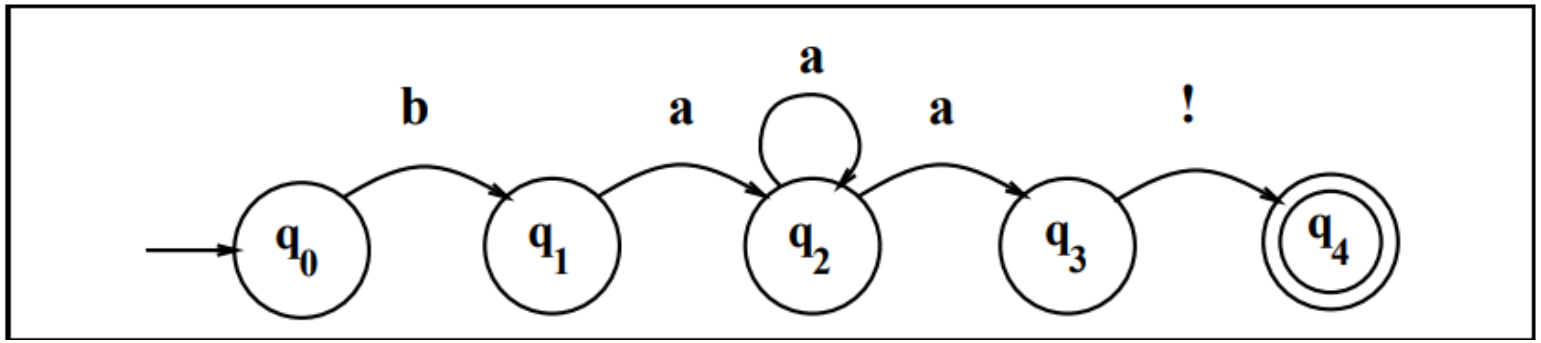
$$L(m) = \{baa!; baaa!; baaaa!; baaaaa!; baaaaaa! \ldots\}$$

An FSA for the words for English numbers 1 – 99



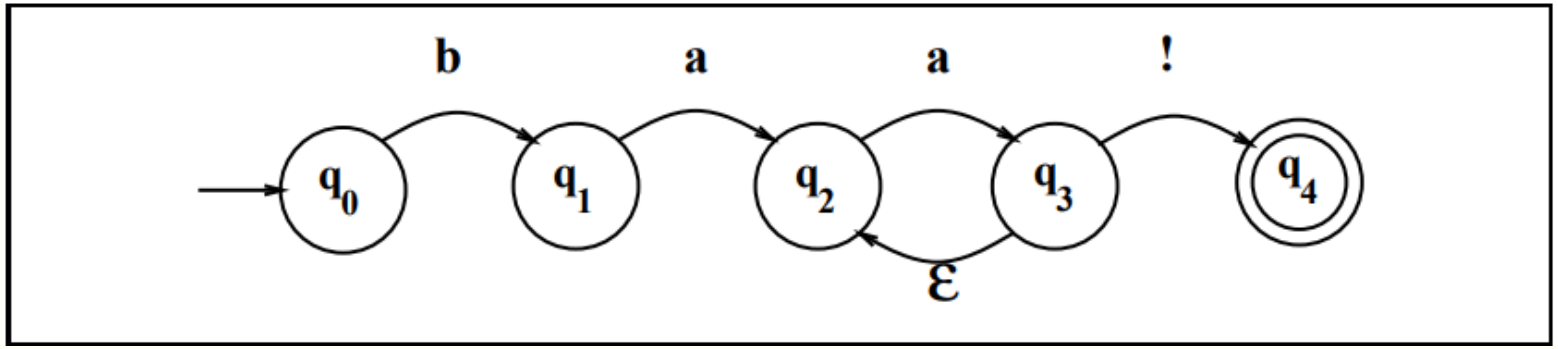
Using an FSA(NDFA or NFA) to Recognize Sheeptalk

baa!
baaa!
baaaa!
baaaaa!
baaaaaa!



A non-deterministic finite-state automaton for talking sheep

Contd…



Another NFSA for the sheep language. It differs from in having an ε-transition

	Input			
State	b	a	!	ε
0	1	∅	∅	∅
1	∅	2	∅	∅
2	∅	2,3	∅	∅
3	∅	∅	4	∅
4:	∅	∅	∅	∅